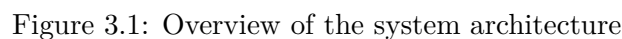


This section introduces the fundamental elements of our approach. We begin by detailing the key concepts used throughout our work. We then present an overview of the proposed architecture, illustrated in Fig. ??, highlighting its main modules and the relationships between infrastructure, services, and orchestration mechanisms. Building on this structural description, we detail the models that characterize the infrastructure and its energy behavior, including the abstraction of computational resources, workload representation, and power dynamics. To facilitate readability and ensure precision in the mathematical formulations that follow, we provide a comprehensive table (Table ??) of notations summarizing the symbols and variables used across this chapter.



Notation	Meaning
N	Set of nodes
S	Set of services
F	Set of CNFs
F_s	Set of CNFs that compose service s
F_n	Set of CNFs running on node n
CPU_f^{req}	Required CPU of CNF f
RAM_f^{req}	Required RAM of CNF f
W_f^{CPU}	CPU workload of CNF f
CPU_n^{alloc}	CPU capacity of node n
RAM_n^{alloc}	RAM capacity of node n
CPU_n	CPU usage of node n (in cores)
$CPU_n^{\%}$	CPU usage of node n (in percent)
PM_n	Power consumption model of node n
P_n	Power consumption of node n
P_n^{green}	Renewable power generation of node n
P_n^{brown}	Non-renewable power consumption of node n
P_n^{wasted}	Wasted power of node n
X	Binary variable for CNF placement
M	Binary variable for CNF migration
B^{CPU}	Binary variable for CPU over-allocation
B^{RAM}	Binary variable for RAM over-allocation
$\widetilde{W}_f^{CPU}$	CPU workload prediction for CNF f
$\widetilde{P}_n^{green}$	Renewable power generation prediction for node n
$\widetilde{G}_n$	Green ratio of node n
$\widetilde{G}_s$	Green ratio of service s
$\widetilde{G}_s^{valid}$	Green ratio constraint satisfaction of service s
G_s^c	Green ratio threshold of service s
SCR_s	Service concentration ratio of service s
m_f	Migration cost of CNF f

Table 3.1: Notation table

3.1.1 Service, Infrastructure & Resources

Our considered system adopts a container-based architecture composed of three main elements:

- *Services*: Logical groupings of CNFs. Each service $s \in S$ is defined by a set of CNFs denoted as F_s .
- *CNFs*: Individual containerized network functions.
- *Nodes*: Physical infrastructure elements with finite computational resources.

We primarily focus on processing (CPU) and memory (RAM) as key resources. Each node N is defined by its allocatable resources $\{CPU_n^{alloc}, RAM_n^{alloc}\}$. For

each CNF $f \in F$, the resource requirements are given as $f = \{CPU_f^{req}, RAM_f^{req}\}$. Regarding CPU, we distinguish between allocated resources and the resources currently in use. Specifically, each CNF f has a time-dependent CPU workload at time t , denoted by $W_f^{CPU}(t)$ and defined in (??).

$$CPU_n^{\%}(t) = \frac{\sum_{f \in F_n} W_f^{CPU}(t)}{CPU_n^{alloc}}. \quad (3.1)$$

3.1.2 Energy

An energy consumption model is associated to each node n and a renewable energy generation model is considered when available on the node.

3.1.2.1 Energy consumption model

We define the power consumption P_n of a node n as a function of its CPU utilization. In our work, the adopted model PM_n (??) is a polynomial of degree m with coefficients a_i obtained via polynomial regression from experimental power data on the platform Grid'5000 [?] as displayed in Fig. ??.

$$PM_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m. \quad (3.2)$$

To create a model correlating CPU usage with power consumption, we designed an experimental method. We executed a CPU-intensive script on a node, simultaneously measuring both CPU utilization and power consumption.

To vary the CPU load, we used **stress-ng**, a Linux-based stress-testing tool. This tool allows precise control over the number of CPU cores used, their load percentage, and the duration of each workload.

CPU usage was measured using the **psutil** Python library (**cpu_percent** function), which provides real-time CPU utilization metrics. On Linux, **psutil** can use the **/proc** filesystem and make calls to **sysinfo**, **getloadavg**, etc.

Power consumption was retrieved using the Grid'5000 API. The platform uses wattmeters to measure node power draw at one-second intervals. All metrics were retrieved post-experiment using the experiment's timestamps.

Our script follows a three-phase approach:

1. *Idle phase*: 30 seconds without load.
2. *Fixed CPU load test*: Sequential stress tests at 10% increments (10%, 20%, ..., 100%) for one minute each, with a 5 seconds cool down, applied across all cores.
3. *Mixed workload test*: Randomized workloads with variable core usage, load percentages, and durations.

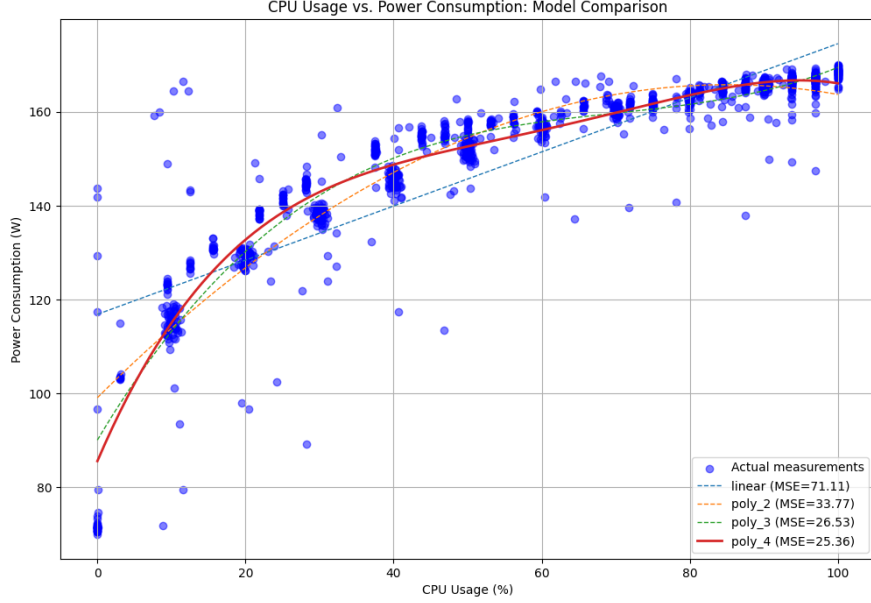


Figure 3.2: Power consumption models obtained via polynomial regression (up to degree 4) from Grid'5000 experiments.

3.1.2.2 Energy generation model

When renewable generation is available on a node n , we define the green power output as a time-varying function $P_n^{green}(t)$. The residual power drawn from the electrical grid (that we call brown power in our work) is then defined by (??).

$$P_n^{brown}(t) = \begin{cases} P_n(t) - P_n^{green}(t) & \text{if } P_n(t) > P_n^{green}(t), \\ 0 & \text{otherwise.} \end{cases} \quad (3.3)$$

In our model, renewable generation can either come from solar or wind sources.

Solar power generation is modeled using a normal distribution-based irradiance profile, estimated via Kernel Density Estimation (KDE) as used in [?]. The instantaneous solar output is then $P_n^{solar}(t) = \eta_n S_n I_n(t)$, where η_n is the panel efficiency, S_n the panel surface area, and $I_n(t)$ the irradiance at time t . The process is detailed in Fig. ??.

Wind power generation model is derived from wind speed generation using a Weibull distribution based on the work in [?]. The final power output can be computed with (??), where P_n^{rated} is the rated power output, $v_n(t)$ the wind speed, and $v_n^{cut_in}$, v_n^{rated} , $v_n^{cut_out}$ the cut-in, rated, and cut-out speeds, respectively. This is also explained by Fig. ?. For both renewable generation models, we add a Gaussian noise to account for natural variability.

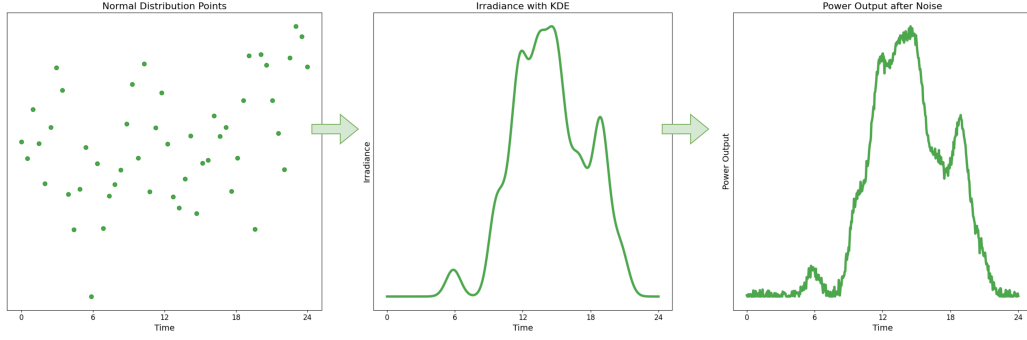


Figure 3.3: Step-by-step visualization of solar power generation from a normal distribution using KDE.

$$P_n^{wind}(t) = \begin{cases} P_n^{rated} \cdot \frac{v_n(t) - v_n^{cut_in}}{v_n^{rated} - v_n^{cut_in}} & \text{if } v_n^{cut_in} \leq v_n(t) < v_n^{rated} \\ P_n^{rated} & \text{if } v_n^{rated} \leq v_n(t) \leq v_n^{cut_out} \\ 0 & \text{otherwise.} \end{cases} \quad (3.4)$$

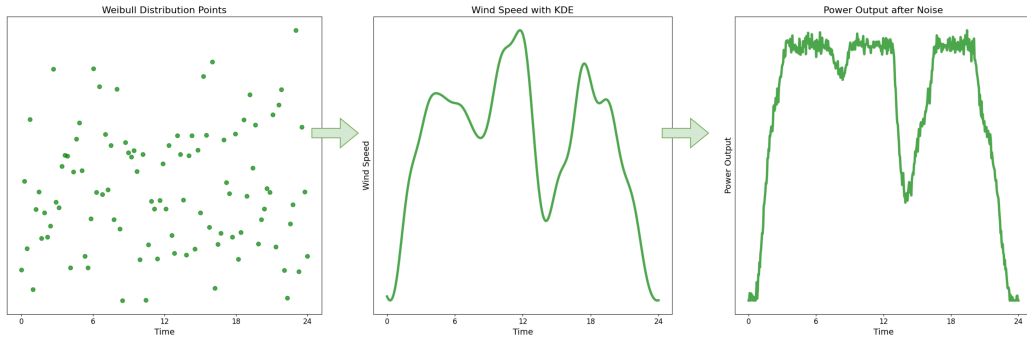


Figure 3.4: Step-by-step visualization of wind power generation from a Weibull distribution using KDE.

This stochastic approach enables us to evaluate our methodology across diverse power generation models, eliminating reliance on a fixed dataset. However, for practical applications, using real-world measurements may be more appropriate.

3.1.3 Monitoring

Monitoring ensures continuous visibility into the state and performance of both physical and virtual entities. It encompasses a wide range of components, from physical nodes to virtual services like our CNFs. Energy-related infrastructure, including power consumption and renewable energy generation, is also closely tracked.

The metrics collected are diverse and purpose-driven. Physical resources like CPU and RAM are monitored alongside virtual resource utilization. Energy metrics, including consumption and generation from renewable sources, are also monitored. Additionally, the system tracks the status and performance of nodes and services, capturing health indicators, uptime, and response times.

The data gathered serves multiple purposes. It provides real-time insights for orchestrators, enabling them to make informed, dynamic decisions. It also acts as a valuable dataset for training machine learning models, supporting predictive analytics and long-term optimization. Finally, this data is transmitted to analysis modules, where it is processed to uncover trends, anomalies, and opportunities for improvement.

3.1.4 Analysis

The analysis module transforms raw monitored data into usable intelligence. By aggregating and correlating metrics across the system, it identifies patterns, detects anomalies, and highlights lack of resources. This processing is not just reactive, it can also be used to make predictions, such as energy availability based on renewable generation trends or anticipating workload demands during peak periods.

These insights directly influence the orchestrator's decisions. For example, predictions about energy shortages or workload spikes can trigger proactive adjustments, ensuring the system remains efficient, reliable, and aligned with operational goals.

3.1.5 Orchestration

Orchestration is the decision-making engine of the system, responsible for dynamically allocating resources and managing services in accordance with predefined policies and real-time insights. It ensures that services are placed optimally, balancing factors such as performance, energy efficiency, and cost. Resource allocation is not static, it evolves based on demand, availability, and policy constraints. A key function of orchestration is the dynamic migration of CNFs.

In our problem, we consider an Edge service orchestration problem where services, composed of multiple CNFs, must be deployed on a set of nodes with limited resources and subject to unpredictable node departures. The objective is to find optimal *service placement* and *service placement reconfiguration* strategies that makes the best use of available energy resources while ensuring service continuity.

3.1.5.1 Service Placement

Service placement orchestrator assigns each CNF to a physical node while satisfying resource constraints and optimizing multiple objectives. We define a binary decision variable $X(n, f, t) \in \{0, 1\}$ that equals 1 when a CNF f is placed on a node n . Each

CNF must be placed on exactly one node, and the total resource requirements of CNFs placed on a node must not exceed its allocatable resources.

3.1.5.2 Service Placement Reconfiguration

We then define service placement reconfiguration as the process of migrating CNFs from a node to another in response to infrastructure changes, such as energy variations or node departures. We use the same variable $X(n, f, t)$ and introduce a new migration variable $M(n, f, t) \in \{0, 1\}$ that equals 1 if CNF f is migrated to node n at time t . The reconfiguration process must also satisfy the same resource constraints.

3.2 Optimization Metrics

We define main optimization metrics for our service placement and service placement reconfiguration problems. The placement problem is only considered at $t + 1$, just after the placement, while the reconfiguration problem can either be considered at $t + 1$ or $t + h$ for the reactive and proactive reconfigurations respectively. To generalize the problem, we consider our metrics at a time $t + h$, where $h \in [1, H]$, with H our prediction horizon.

3.2.1 Overall power consumption

We compute the CPU usage of each node at time $t + h$ as in (??) and (??), from the prediction of CNF f workload $\widetilde{W}_f^{CPU}$. Then, in (??), we derive the power consumption using the power model of the node.

$$\widetilde{CPU}_n(t + h) = \sum_{f \in F} X(n, f, t + h) \cdot \widetilde{W}_f^{CPU}(t + h). \quad (3.5)$$

$$\widetilde{CPU}_n^{\%}(t + h) = \min \left(1, \frac{\widetilde{CPU}_n(t + h)}{CPU_n^{alloc}} \right). \quad (3.6)$$

$$\widetilde{P}_n(t + h) = PM_n \left(\widetilde{CPU}_n^{\%}(t + h) \right). \quad (3.7)$$

3.2.2 Brown power consumption

The brown power consumption is then obtained in (??) by subtracting the prediction of power generation $\widetilde{P}_n^{green}$ to the estimated power consumption.

$$\tilde{P}_n^{brown}(t+h) = \begin{cases} \tilde{P}_n(t+h) - \tilde{P}_n^{green}(t+h) & \text{if } \tilde{P}_n(t+h) > \tilde{P}_n^{green}(t+h), \\ 0 & \text{otherwise.} \end{cases} \quad (3.8)$$

3.2.3 Wasted power

Similarly, the wasted power can be obtained using (??) by subtracting the estimated power consumption to the prediction of power generation $\tilde{P}_n^{green}$.

$$\tilde{P}_n^{wasted}(t+h) = \begin{cases} \tilde{P}_n^{green}(t+h) - \tilde{P}_n(t+h) & \text{if } \tilde{P}_n(t+h) < \tilde{P}_n^{green}(t+h), \\ 0 & \text{otherwise.} \end{cases} \quad (3.9)$$

3.2.4 Service green ratio

From both the prediction of green power generation $\tilde{P}_n^{green}$ and our estimation of power consumption $\tilde{P}_n$ in (??), we can compute the *green ratio* of a node n at time $t+h$ as in (??), that represents the percentage of the energy consumed on the node that is green at time slot $t+h$. From this ratio, we define the green ratio of a service s as (??), assuming that the green ratio of a CNF is the ratio of the node it is placed on. The satisfaction of the green ratio constraint for a service is then defined in (??), where G_s^c is the minimum required green ratio for service s . Since we aim to avoid interruptions in our system, defining an acceptable threshold of green energy usage for our services is an alternative to purely maximize the green usage of our infrastructure. This constraint could be negotiated between the service provider and the infrastructure provider, that orchestrate the wanted services.

$$\tilde{G}_n(t+h) = \begin{cases} \frac{\tilde{P}_n^{green}(t+h)}{\tilde{P}_n(t+h)} & \text{if } \tilde{P}_n^{green}(t+h) < \tilde{P}_n(t+h) \\ 1 & \text{otherwise.} \end{cases} \quad (3.10)$$

$$\tilde{G}_s(t+h) = \frac{1}{|F_s|} \sum_{f \in F_s} \sum_{n \in N} X(n, f, t+h) \cdot \tilde{G}_n(t+h). \quad (3.11)$$

$$\tilde{G}_s^{valid}(t+h) = \begin{cases} 1 & \text{if } \tilde{G}_s(t+h) \geq G_s^c, \\ 0 & \text{otherwise.} \end{cases} \quad (3.12)$$

3.2.5 Service concentration ratio

To minimize the impact of node departures on service continuity, we define the *service concentration ratio (SCR)* of a service s at time $t+h$ as in (??), where $|F_s|$

is the total number of CNFs of service s and $n_s(t)$ is the highest number of CNFs that on the same node at time t . The goal of this metric is to minimize the amount of CNFs on the same node in case of departure. $r \in \{0, 1\}$ is a constant to define a threshold that is acceptable, e.g. 0.5 for a maximum of 50% of the service's CNFs on the same node. The idea behind this metric is to avoid clustering too much CNFs of the same service on the same node, that will cause an interruption of the service in case of departure. This objective is contradictory with the service green ration satisfaction, that would prioritize the choice of the “best” node, i.e. the greenest. Note that the SCR metric is different from load balancing, since we still can have large difference in the load of our nodes.

$$SCR_s(t+h) = \begin{cases} r \cdot \frac{|F_s|}{n_s(t+h)} & \text{if } n_s(t+h) > r \cdot |F_s| \\ 1 & \text{otherwise.} \end{cases} \quad (3.13)$$

3.2.6 Migration cost

Finally, we define the migration cost as in (??) for each migrated CNF $f \in \hat{F}$, where $t_{init}(f)$ is the initialization time of a CNF f . Our logic behind this is that to preserve the continuity of the service, the original instance of the migrated CNF must continue to run until the new instance is initialized, leading to a surplus of consumption during this migration.

$$m_f(t+h) = \sum_{n \in N} M(n, f, t+h) \cdot t_{init}(f) \cdot CPU_f^{req}. \quad (3.14)$$

Algorithm 1: Greedy Placement

Input: N : Nodes, F_s : CNFs to place, X : Placement Matrix

Output: Placement mapping at $t + 1$

```

1  $N^{(\tau)} \leftarrow \text{sort}(N, P_n^{\text{green}} - P_n);$ 
2 for  $f \in F_s$  do
3   for  $n \in N^{(\tau)}$  do
4     if  $n$  has enough resources for  $f$  then
5        $X(n, f, t + 1) \leftarrow 1;$ 
6       break;
7   if  $\sum_{n \in N} X(n, f, t + 1) = 0$  then
8      $X(0, f, t + 1) \leftarrow 1;$ 
9 return  $X;$ 

```

Algorithm 2: GREENING (migration)

Input: N : Nodes, F : Running CNFs, X : Placement Matrix, M :
Migration Matrix

Output: Migration plan at $t + 1$

```

1  $N^{(\tau)} \leftarrow \text{sort}(N, P_n^{\text{green}} - P_n);$ 
2 for  $n \in N^{(\tau)}$  do
3   if  $P_n^{\text{green}}(t) > P_n^{\text{green}}(t + 1)$  then
4      $\text{candidates} \leftarrow \{f \in F, X(n, f, t) = 1\};$ 
5     while  $P_n(t + 1) > P_n^{\text{green}}(t + 1)$  do
6       if  $\text{candidates} = \emptyset$  then
7         break;
8        $\text{sorted\_candidates} \leftarrow \text{sort}(\text{candidates}, t_{\text{init}}(f) \cdot \text{CPU}_f^{\text{req}});$ 
9       for  $d \in N^{(\tau)}$  do
10        if  $d = n$  then
11          continue;
12        if  $\text{sorted\_candidates}[0]$  can be placed on  $d$  and new
            $P_n(t + 1) \leq P_n^{\text{green}}(t + 1)$  then
13           $X(d, f, t + 1) \leftarrow 1;$ 
14           $X(n, f, t + 1) \leftarrow 0;$ 
15           $M(d, f, t + 1) \leftarrow 1;$ 
16        Remove  $\text{sorted\_candidates}[0]$  from  $\text{candidates};$ 
17 return  $M;$ 

```

Algorithm 3: Lb-diversity

Input: N : Nodes, F_s : CNFs to place, X : Placement Matrix
Output: Placement mapping at $t + 1$

```

1  $\bar{F} \leftarrow \text{sort } F_s \text{ by sum of CPU and RAM ranking;}$ 
2 for  $f \in \bar{F}$  do
3    $\text{candidates} \leftarrow \emptyset;$ 
4    $m \leftarrow -\infty;$ 
5   for  $n \in N$  do
6      $n \leftarrow \sum_{k \in F} X(n, k, t);$ 
7     if  $n < m$  then
8        $\text{candidates} \leftarrow \{n\};$ 
9     else if  $n = m$  then
10       $\text{candidates} \leftarrow \text{candidates} \cup \{n\};$ 
11  $\bar{N} \leftarrow \text{sort } \text{candidates} \text{ by sum of CPU and RAM usage;}$ 
12 for  $n \in \bar{N}$  do
13   if  $f \text{ can be placed on } n$  then
14      $X(n, f, t + 1) = 1;$ 
15 return  $X;$ 

```

Algorithm 4: Traditional Genetic Algorithm

Input: P : Population size, G : Generations, μ : Mutation probability, σ :
 Selection percentage
Output: Best individual

```

1 Initialize random population;
2 Evaluate population;
3  $g = 0;$ 
4 while Termination criterion not met or  $g < G$  do
5    $g = g + 1;$ 
6   Select  $\sigma$  of best chromosomes of population;
7   Perform crossover;
8   Perform mutation with each gene having a probability of  $\mu$  to mutate;
9   Evaluate the new population;
10 return Best individual of population;

```

Algorithm 5: Non-dominated Sorting Genetic Algorithm II (NSGA-II)

Input: P : Population size, G : Generations, μ : Mutation probability
Output: Best Pareto Front

3.3 Algorithms

3.3.1 Heuristics

3.3.1.1 GREENING

3.3.1.2 Lb-diversity

3.3.2 Genetic Algorithms

3.3.2.1 Chromosomes

3.3.2.2 Traditional Genetic Algorithm